

ECO-3D Project Overview

Date: March 15, 2026

1. Project Overview

ECO-3D is a full-stack geospatial AI platform that transforms a map click (latitude/longitude) into a sustainability and feasibility assessment for land development. It combines environmental analytics, flood-risk estimation, buildability scoring, and AI-assisted floor-plan generation.

Primary users include architects, urban planners, real-estate advisory teams, and sustainability consultants.

End-to-end workflow:

1. User selects a location in the web map.
2. Backend detects parcel boundary (or safe fallback).
3. System fetches real-time environmental data from multiple public APIs.
4. AI and physics-based models compute flood risk and buildability.
5. Platform generates floor-plan variants and stores analysis results.
6. Frontend renders analysis dashboards, floor plans, and report outputs.

2. Core Data Sources

The runtime system integrates public APIs (no keys required for most calls):

- Open-Elevation (elevation/slope context)
- Open-Meteo Forecast and Climate (wind/rainfall)
- SoilGrids (soil composition and chemistry)
- NASA POWER (solar and NDVI proxy)
- Open-Meteo Flood API / GloFAS (river discharge context)
- OSM Overpass (distance to water and boundary-related lookups)

Training/evaluation datasets in this repository:

- data/flood_training.csv (2000 rows)
- data/buildability_training.csv (3000 rows)
- data/segmentation_labels.csv (500 rows)

3. System Architecture

Frontend:

- Next.js 14 + React 18 + TypeScript
- Leaflet map UI, state management via Zustand

Backend:

- FastAPI (Python)
- Async orchestration with asyncio and httpx
- Route modules for analysis, floorplan, boundary, report, auth, notifications

Persistence:

- SQLAlchemy async ORM
- SQLite default for local development; PostgreSQL supported for production

AI/Analytics Layers:

1. Segmentation (DeepLabV3 wrapper, with fallback)
2. Tree detection (YOLOv8 wrapper, with fallback)
3. Environmental feature aggregation from live APIs
4. Flood scoring (physics blend + optional ML path)
5. Buildability scoring (physics blend + optional ML path)
6. Floorplan generation and scoring with variant selection

4. Data Flow Summary

1. UI sends /plot-boundary request for selected coordinates.
2. UI sends /analyze-plot with plot_id + coordinates (+ optional polygon).
3. Backend runs concurrent segmentation, tree detection, and environmental fetches.
4. Backend computes flood and buildability scores.
5. Analysis is persisted to database and returned to frontend.
6. UI requests /generate-floorplan with area/preferences.
7. Floorplan service returns best variant + detailed geometry.
8. UI can fetch /report/{plot_id} for consolidated output.

5. Important Design Decisions

- Crash-resistant behavior: every critical external dependency has deterministic fallback logic.
- Async-first integration: multi-API calls are executed concurrently to reduce latency.
- Hybrid intelligence: deterministic formulas remain available even when ML weights are missing.
- Progressive enhancement: local setup works with SQLite and fallback models; production can scale to PostgreSQL and trained weights.

6. Key Trade-offs and Limitations

- Public API latency can vary by region and time.

- Synthetic fallback outputs are stable but less precise than fully live + trained execution.
- Some models are optional and may train on synthetic data unless enriched with curated real labels.
- Legal/land-record integrations vary by jurisdiction and data availability.

7. How to Run (Quick Commands)

Backend:

- `cd backend`
- `python -m venv .venv`
- `.venv\Scripts\activate`
- `pip install -r requirements.dev.txt`
- `uvicorn main:app --reload --host 0.0.0.0 --port 8000`

Frontend:

- `cd frontend`
- `npm install`
- `set NEXT_PUBLIC_API_URL=http://localhost:8000`
- `npm run dev`

Data + model workflow:

- `python scripts/generate_synthetic_data.py`
- `python scripts/train_flood_model.py`
- `python scripts/train_buildability_model.py`
- `python scripts/train_segmentation_model.py`
- `python scripts/evaluate_models.py`

8. Repository Highlights

- `backend/services/analysis_pipeline.py`: main analysis orchestration
- `backend/services/real_env_data.py`: real-time feature aggregation and core scoring formulas
- `backend/services/floorplan_service.py`: floorplan generation and eco scoring
- `frontend/app/map/page.tsx`: map interaction and analysis trigger
- `frontend/lib/api.ts`: typed client contracts for backend APIs

9. Conclusion

ECO-3D provides an integrated platform for environmental feasibility and AI-assisted architectural pre-planning. Its most notable engineering characteristic is resilient operation under partial failure: it favors graceful degradation over hard runtime crashes, making it practical for exploratory planning workflows where data reliability can vary by location.